



1. Dynamic Components

```
//Ts
@ViewChild('container', { read: ViewContainerRef }) container!: ViewContainerRef;

loadComponent() {
  this.container.createComponent(MyDynamicComponent);
}

//HTML
<div #container></div>
```

Explanation :

1. Normally, components are declared in templates.
2. With dynamic components, you can create and insert components at runtime.
3. Use `ViewContainerRef` + `ComponentFactoryResolver` (Angular <14) or just `createComponent()` (Angular 14+).



2. ViewChild & ContentChild

```
//Ts
@ViewChild('myDiv') myDiv!: ElementRef;
@ContentChild('projected') projected!: ElementRef;

ngAfterViewInit() {
  console.log(this.myDiv.nativeElement.innerText);
}

//HTML
<!-- parent -->
<app-child>
  <p #projected>Projected Text</p>
</app-child>

<!-- child -->
<div #myDiv>Child View</div>
<ng-content></ng-content>
```

Explanation :

1. ViewChild = Access a child component or DOM element inside the component's own template.
2. ContentChild = Access projected content passed using <ng-content>.



3. Angular Elements

```
//Ts
import { createCustomElement } from '@angular/elements';

const el = createCustomElement(MyComponent, { injector });
customElements.define('my-element', el);

//HTML
<my-element></my-element>
```

Explanation :

1. Allows you to package Angular components as native Web Components.
2. They can then be used in any framework (React, Vue) or plain HTML.



4. Content Projection

```
//HTML
<!-- child -->
<ng-content select=".header"></ng-content>
<ng-content select=".body"></ng-content>

<!-- parent -->
<app-card>
  <h2 class="header">Title</h2>
  <p class="body">Content goes here</p>
</app-card>
```

Explanation :

1. Lets you project content from parent to child.
2. Multi-slot projection allows multiple content placeholders.



5. Custom Event Emitters

```
//Ts
// child
@Output() clicked = new EventEmitter<string>();

onClick() {
  this.clicked.emit('Button was clicked!');
}

//HTML
<!-- parent -->
<app-child (clicked)="onChildClick($event)"></app-child>
```

Explanation :

1. Used for parent-child communication.
2. Child component emits events, parent listens.



6. Renderer2 & ElementRef

```
//Ts
@ViewChild('myDiv') div!: ElementRef;

constructor(private renderer: Renderer2) {}

ngAfterViewInit() {
  this.renderer.setStyle(this.div.nativeElement, 'color', 'red');
}

//HTML
<div #myDiv>Hello</div>
```

Explanation :

1. Used for translating Angular apps into multiple languages.
2. Angular provides the i18n attribute + extraction tools.
3. Translations can then be provided for multiple locales.



7. Internationalization (i18n)

```
//HTML
<h1 i18n="site welcome message">Welcome to my app!</h1>
```

Explanation :

1. Used for translating Angular apps into multiple languages.
2. Angular provides the i18n attribute + extraction tools.
3. Translations can then be provided for multiple locales.

8. Angular Universal (SSR)

Explanation :

1. Enables Server-Side Rendering (SSR) for Angular apps.
2. Benefits:
3. Faster first paint (SEO friendly)
4. Better performance on slow devices

```
bash: ng add @nguniversal/express-engine
```




9. Micro Frontends

```
//Ts
loadRemoteModule({
  type: 'module',
  remoteEntry: 'http://localhost:4201/remoteEntry.js',
  exposedModule: './ProductModule'
});
```

Explanation :

1. Break a large app into smaller, independently deployable pieces.
2. Webpack Module Federation helps load features from remote apps dynamically.



10. Signals

```
//Ts
import { signal, computed, effect } from '@angular/core';

count = signal(0);
double = computed(() => this.count() * 2);

constructor() {
  effect(() => console.log('Count is', this.count()));
}

increment() {
  this.count.set(this.count() + 1);
}
```

Explanation :

1. New way to handle reactive state (alternative to RxJS in many cases).
2. Signals are reactive variables that notify dependents automatically.